

Amazon Kindle Store Review Sentiment Analysis

About the Dataset

This project uses a 5-core dataset of Amazon Kindle Store product reviews, spanning from May 1996 to July 2014, containing 12000 entries. Each reviewer has reviewed at least 5 products, and each product has received at least 5 reviews.

Dataset Columns:

0 Unnamed: 0.1 12000 non-null int64
1 Unnamed: 0 12000 non-null int64
2 asin 12000 non-null object
3 helpful 12000 non-null object
4 rating 12000 non-null int64
5 reviewText 12000 non-null object
6 reviewTime 12000 non-null object
7 reviewerID 12000 non-null object
8 reviewerName 11962 non-null object
9 summary 11998 non-null object
10 unixReviewTime 12000 non-null int64

Source:

Amazon Product Data by Julian McAuley, UCSD

<http://jmcauley.ucsd.edu/data/amazon>

Project Goals:

-  Classify reviews as positive or negative.
-  Use NLP techniques to clean and understand text.
-  Handle class imbalance (more positives than negatives).
-  Evaluate model with accuracy, recall, F1-score.
-  Visualize textual and sentiment patterns.

Data Cleaning Steps:

- ✓ Removed null values in reviewText, overall
- ✓ Dropped neutral reviews (rating == 3)
- ✓ Labeled:
 - Positive = 4 or 5 stars
 - Negative = 1 or 2 stars
- ✓ Created a sentiment column (binary: 1/0)

□ Text Preprocessing (Advanced NLP Pipeline):

def clean_text(text):

```
text = BeautifulSoup(str(text), "html.parser").get_text() # Remove HTML
text = text.lower() # Convert to lowercase
text = re.sub(r"http\S+|www\S+|https\S+", "", text) # Remove URLs
text = re.sub(r"[^a-z0-9\s.,!?!]", "", text) # Keep letters/numbers/punctuation
text = re.sub(r"\s+", " ", text).strip() # Remove extra spaces
return text
```

Tools used:

- nltk → for stopwords, lemmatization, tokenization
- re → regular expressions
- WordNetLemmatizer

Applied this to every review to generate cleaned_review.

Sentiment Polarity using TextBlob:

What is TextBlob?

TextBlob is a Python library for processing textual data. It is built on top of NLTK and Pattern and provides simple APIs for common NLP tasks, including:

- Part-of-speech tagging
- Noun phrase extraction
- Sentiment analysis 
- Text classification
- Translation, and more.

 You used it for Sentiment Analysis in your Amazon Reviews project.

What is Sentiment Polarity?

TextBlob's sentiment analyzer returns:

1. Polarity

- A float value in the range [-1.0, 1.0]
- Interpreted as:
 - -1.0 → Most negative
 - **** 0.0 **** → Neutral
 - +1.0 → Most positive

2. Subjectivity (*Optional*)

- A float in [0.0, 1.0]
- Measures how subjective or opinion-based a sentence is.
 - 0.0 → very objective
 - 1.0 → very subjective

But for your case, only polarity was used.

How TextBlob Calculates Sentiment

TextBlob uses a lexicon-based approach, specifically the Pattern Analyzer, which:

- Contains a dictionary of words with predefined polarity scores.
- For example:

- "amazing" → +0.8
- "terrible" → -0.9
- "book" → 0 (neutral)

How it works internally:

1. Tokenizes the input text into words.
2. Looks up polarity for each word.
3. Applies rules for negations (e.g., "not good" → negative).
4. Computes the average polarity of all tokens.

□ **Example in Python:**

```
from textblob import TextBlob
```

```
text = "This book is absolutely amazing!"
```

```
blob = TextBlob(text)
```

```
print(blob.sentiment.polarity) # Output: e.g., 0.9 (very positive)
```

For a full DataFrame:

```
df['polarity'] = df['reviewText'].apply(lambda x: TextBlob(x).sentiment.polarity)
```

☑ **Application in Your Project**

You applied this to each review to:

- Measure how positive or negative each review was.
- Visualize overall sentiment trends.
- Check if TextBlob polarity matches the actual star ratings.
- Understand review behavior before training ML models.

🔗 **Pros and Cons of TextBlob Sentiment Analysis**

☑ **Pros:**

- Very easy to use (1 line of code).
- Works well for short texts like reviews.

- **Lightweight and no training needed.**

✗ Cons:

- **Doesn't handle context well (e.g., sarcasm).**
 - **Not state-of-the-art (compared to BERT or transformers).**
 - **Only good for basic sentiment tasks.**
-

🏠 When to Use It?

Use TextBlob:

- **As a baseline sentiment score.**
- **For preprocessing or feature engineering.**
- **To get quick insights before diving into ML.**

For production-grade systems or nuanced emotion detection, consider:

- **VADER (for social media tone)**
- **BERT-based models (for contextual understanding)**

Feature Extraction Using TF-IDF

TF-IDF is a numerical statistic that reflects how important a word is to a document in a collection (corpus).

It is widely used in text mining, information retrieval, and NLP to convert text data into numeric feature vectors that machine learning algorithms can understand.

$$\square \text{ TF-IDF} = \text{TF} \times \text{IDF}$$

Let's break it down:

$\square$ Term Frequency (TF)

How frequently a term appears in a document.

$\square$ Inverse Document Frequency (IDF)

Measures how unique or rare a word is across all documents.

+ Combining both:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t) \quad \text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Why Use TF-IDF?

- Highlights important terms in a review (e.g., "excellent", "boring") while ignoring common ones like "the", "and", etc.
 - Unlike Bag-of-Words (BoW), it reduces the weight of frequent but uninformative terms.
 - Produces sparse, high-dimensional vectors suitable for ML.
-

TF-IDF in my Project:

Applied TF-IDF to transform the reviewText column into numerical features for training classifiers.

Code Used:

python

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
```

```
X = tfidf.fit_transform(df['reviewText_clean'])
```

Explanation of Parameters:

- **max_features=5000:** Keeps top 5,000 most important words.
 - **stop_words='english':** Removes common English stop words like “is”, “and”, “the”.
 - **fit_transform:** Fits the model to the text and transforms it into a matrix of TF-IDF features.
-

Output of TF-IDF

This results in a sparse matrix of shape (n_reviews, 5000).

Each row → one review

Each column → one of the 5,000 selected words

Each cell → the TF-IDF score of the word in that review

Balancing Classes with SMOTE

Why SMOTE?

My dataset had far more positive reviews.

SMOTE (Synthetic Minority Over-sampling Technique) generates new samples for the minority class to balance the dataset.

python

```
from imblearn.over_sampling import SMOTE
```

```
X_resampled, y_resampled = SMOTE().fit_resample(X, y)
```

This ensures my model doesn't just predict “positive” for everything.

Model Training: Logistic Regression

Why Logistic Regression?

- Fast and scalable for large, sparse data
- Performs well with TF-IDF input

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

You split your data:

python

CopyEdit

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(...)
```

Model Evaluation

Classification Report (94% accuracy):

markdown

CopyEdit

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

negative	0.92	0.97	0.94	2023
----------	------	------	------	------

positive	0.97	0.91	0.94	2013
----------	------	------	------	------

 Final Accuracy: 94.05%

Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative 1959		64
Actual Positive 176		1837

Visualizations You Used

1. Review Length vs Sentiment Polarity

2. Word Clouds:

- Positive → words like “amazing”, “love”, “great”
- Negative → “boring”, “waste”, “disappointed”

GloVe (Global Vectors for Word Representation) is a popular unsupervised machine learning algorithm used for obtaining word embeddings. It creates dense vector representations of words based on their global context in a corpus of text. Here's a detailed breakdown of GloVe:

1. What is GloVe?

GloVe is a vector representation of words that captures their meaning by looking at the global co-occurrence statistics of a corpus. In simple terms, GloVe creates word vectors by analyzing how often words appear near each other in large text corpora.

Unlike Word2Vec, which only focuses on local context (i.e., words within a small window), GloVe takes into account global word-word co-occurrence across the entire corpus, making it more robust.

2. Why Use GloVe?

- **Capturing Global Context:** GloVe captures both local and global word co-occurrences, making it better at capturing semantic relationships.
- **Pre-trained Models:** GloVe provides pre-trained embeddings on large corpora like Wikipedia, Common Crawl, and others. These embeddings can be used for various NLP tasks without the need to train them from scratch.
- **Efficiency:** GloVe is computationally efficient because it avoids scanning the entire context of words, unlike Word2Vec which focuses on nearby context.

3. Word Vectors & Semantic Relationships:

- One of the key strengths of GloVe is that word vectors that are close in the vector space are semantically similar. For example, the word vectors for "king" and "queen" will be close, and the relationship between "man" and "woman" can be captured through vector arithmetic:

$$\text{king} - \text{man} + \text{woman} = \text{queen} \quad \text{vector}\{\text{king}\} - \text{vector}\{\text{man}\} + \text{vector}\{\text{woman}\} = \text{vector}\{\text{queen}\}$$

4. Applications of GloVe:

- **Text Classification:** GloVe embeddings can be used as input features in classification models, such as sentiment analysis.
- **Similarity and Analogy Tasks:** GloVe embeddings allow for measuring the similarity between words or finding word analogies.
- **Clustering:** GloVe vectors can be clustered to group semantically similar words together.
- **Information Retrieval:** GloVe can improve search engines by better understanding the semantic meaning of search queries and documents.

5. Advantages of GloVe:

- **Word Relationships:** The word vectors obtained from GloVe reflect strong semantic and syntactic relationships.

- **Training Efficiency:** It's less computationally expensive compared to other models that require extensive training data, such as BERT.
- **Scalability:** GloVe scales well to large datasets.

6. GloVe vs Word2Vec:

- **Word2Vec:** Uses local context to predict words in a given window (skip-gram or CBOW models). It doesn't explicitly use co-occurrence counts, making it more suited for capturing semantic relationships from local word contexts.
- **GloVe:** Uses global co-occurrence statistics and factorizes the matrix of word co-occurrences, which helps it capture global word relationships and similarities better than Word2Vec.

7. Limitations of GloVe:

- **Fixed Embeddings:** GloVe creates static embeddings, meaning they don't adapt to the context of the word in a sentence (unlike BERT).
- **Vocabulary Size:** If a word is out-of-vocabulary, it cannot be represented, unlike contextual embeddings from models like BERT.
- **Computationally Intensive:** The co-occurrence matrix generation can be computationally expensive, especially with large corpora.

8. GloVe Embeddings in Action:

- You can use GloVe embeddings in NLP tasks by leveraging pre-trained word vectors or by training your own on a specific corpus.
- For instance, in sentiment analysis, you can use GloVe word vectors as input features into machine learning models to classify sentiments in text.

Final Report: Sentiment Analysis using TF-IDF, GloVe Embeddings, and Logistic Regression

1. Project Overview

In this project, the goal was to perform sentiment analysis on a large dataset of product reviews, categorizing them into two classes: positive and negative. The analysis aimed to explore the effectiveness of different text representation techniques (TF-IDF and GloVe embeddings) and evaluate the performance of a Logistic Regression model for sentiment classification.

2. Dataset

- The dataset used consists of customer reviews and ratings. The reviews are textual data, while the ratings are numerical.
- Preprocessing Steps:
 - Removed HTML tags and irrelevant characters from the text using BeautifulSoup.
 - Lowercased the text and eliminated URLs and special characters using regular expressions.

3. Text Representation Techniques

The dataset was represented using two different text representation techniques:

- TF-IDF (Term Frequency-Inverse Document Frequency):
 - TF-IDF measures the importance of a word within a document relative to the frequency of the word across all documents.
 - It is effective at capturing important words and their context within the document. The top 5000 features were selected for vectorization.
- GloVe (Global Vectors for Word Representation):
 - GloVe embeddings capture the semantic meaning of words by representing them as vectors in a high-dimensional space.
 - Pre-trained GloVe embeddings (available in 50, 100, 200, and 300-dimensional vectors) were used to represent the words in the reviews.
 - The average word embedding of each review was computed to create a vector for input to the machine learning model.

4. Model: Logistic Regression

- Logistic Regression was chosen as the classification algorithm because it is simple, effective, and interpretable.
- The model was trained on both the TF-IDF and GloVe embeddings representations of the review data.

- **SMOTE (Synthetic Minority Over-sampling Technique)** was applied to balance the dataset, addressing any class imbalance in the positive and negative classes.

5. Evaluation Metrics

The model was evaluated using the following metrics:

- **Precision:** Measures the accuracy of positive predictions.
- **Recall:** Measures the ability of the model to correctly identify positive instances.
- **F1-Score:** The harmonic mean of precision and recall, giving a balanced measure of performance.
- **Accuracy:** The overall correctness of the model in predicting the sentiment.

6. Performance Results

TF-IDF Model:

- **Precision:**
 - **Negative: 0.76**
 - **Positive: 0.79**
- **Recall:**
 - **Negative: 0.80**
 - **Positive: 0.74**
- **F1-Score:**
 - **Negative: 0.78**
 - **Positive: 0.76**
- **Accuracy: 77.13%**

Confusion Matrix for TF-IDF:

[[1614 409]

[514 1499]]

- **True Negatives (TN): 1614**
- **False Positives (FP): 409**
- **False Negatives (FN): 514**
- **True Positives (TP): 1499**

GloVe Model:

- **Precision:**
 - **Negative: 0.76**

- **Positive: 0.78**
- **Recall:**
 - **Negative: 0.79**
 - **Positive: 0.72**
- **F1-Score:**
 - **Negative: 0.77**
 - **Positive: 0.75**
- **Accuracy: 76.50%**

Confusion Matrix for GloVe:

[[1587 436]

[574 1439]]

- **True Negatives (TN): 1587**
- **False Positives (FP): 436**
- **False Negatives (FN): 574**
- **True Positives (TP): 1439**

7. Insights

- **TF-IDF performed slightly better than GloVe in terms of accuracy, recall, and F1-score.**
- **The model showed a good balance between precision and recall, but there were still false positives and false negatives that could be further addressed.**
- **Recall for the negative class was better than for the positive class, suggesting that the model has a bias toward predicting negative reviews. The F1-score for the negative class was also higher.**

8. Recommendations for Improvement

- **Hyperparameter tuning:** Try adjusting the regularization strength in Logistic Regression, or experiment with other models like SVMs or Random Forests for better performance.
- **Class Imbalance:** Although SMOTE helped balance the dataset, experimenting with class weights in Logistic Regression could further improve the model's performance.
- **Advanced Techniques:** For better understanding and prediction, consider using BERT or GPT-based models for fine-grained sentiment analysis, as these deep learning models often outperform traditional approaches like TF-IDF and GloVe.

9. Conclusion

This project demonstrated that both TF-IDF and GloVe embeddings are effective for sentiment analysis. The Logistic Regression model achieved a solid accuracy of around 77%, with room for improvement through advanced techniques and further tuning.